

Simple Messaging Application in Python

Here's a complete messaging application built with Python using sockets for network communication and Tkinter for the GUI interface.

Server Code (messaging_server.py)

```
import socket
import threading
import json
from datetime import datetime

class MessagingServer:
    def __init__(self, host='0.0.0.0', port=5555):
        self.host = host
        self.port = port
        self.server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server.bind((host, port))
        self.server.listen()
        self.clients = {}
        self.message_history = []

        print(f"Server started on {host}:{port}")

    def broadcast(self, message, sender=None):
        """Send message to all connected clients"""
        message_json = json.dumps(message)
        for client_socket in self.clients:
            if client_socket != sender: # Don't send back to sender
                try:
                    client_socket.send(message_json.encode('utf-8'))
                except:
                    self.remove_client(client_socket)

    def handle_client(self, client_socket, address):
        """Handle individual client connections"""
        try:
            # Get username from first message
            username_msg = client_socket.recv(1024).decode('utf-8')
            username = json.loads(username_msg)['username']
            self.clients[client_socket] = username

            # Send welcome message
```

```

welcome = {
    'type': 'system',
    'content': f"{username} joined the chat",
    'timestamp': str(datetime.now())
}
self.broadcast(welcome)

# Send message history to new client
if self.message_history:
    history_msg = {
        'type': 'history',
        'content': self.message_history
    }
    client_socket.send(json.dumps(history_msg).encode('utf-8'))

# Handle incoming messages
while True:
    message = client_socket.recv(1024).decode('utf-8')
    if not message:
        break

    message_data = json.loads(message)
    message_data['timestamp'] = str(datetime.now())
    self.message_history.append(message_data)

    self.broadcast(message_data, client_socket)

except Exception as e:
    print(f"Error with client {address}: {e}")
finally:
    self.remove_client(client_socket)

def remove_client(self, client_socket):
    """Remove disconnected client"""
    if client_socket in self.clients:
        username = self.clients[client_socket]
        del self.clients[client_socket]
        client_socket.close()

    leave_msg = {
        'type': 'system',
        'content': f"{username} left the chat",
        'timestamp': str(datetime.now())
    }
    self.broadcast(leave_msg)
    print(f"{username} disconnected")

```

```

def start(self):
    """Start accepting connections"""
    while True:
        client_socket, address = self.server.accept()
        print(f"New connection from {address}")

        client_thread = threading.Thread(
            target=self.handle_client,
            args=(client_socket, address)
        )
        client_thread.start()

if __name__ == "__main__":
    server = MessagingServer()
    server.start()

```

Client Code (messaging_client.py)

```

import socket
import threading
import json
from tkinter import *
from tkinter import scrolledtext, messagebox
from datetime import datetime

class MessagingClient:
    def __init__(self, root):
        self.root = root
        self.root.title("Python Messenger")
        self.root.geometry("600x500")

        # Connection variables
        self.client_socket = None
        self.connected = False
        self.username = ""

        # Create GUI
        self.create_connection_frame()
        self.create_chat_frame()

        # Initially show connection frame
        self.show_connection_frame()

    def create_connection_frame(self):

```

```

"""Create the connection setup interface"""
self.connection_frame = Frame(self.root)

Label(self.connection_frame, text="Server IP:").pack(pady=5)
self.server_ip_entry = Entry(self.connection_frame)
self.server_ip_entry.insert(0, "localhost")
self.server_ip_entry.pack(pady=5)

Label(self.connection_frame, text="Port:").pack(pady=5)
self.port_entry = Entry(self.connection_frame)
self.port_entry.insert(0, "5555")
self.port_entry.pack(pady=5)

Label(self.connection_frame, text="Username:").pack(pady=5)
self.username_entry = Entry(self.connection_frame)
self.username_entry.pack(pady=5)

self.connect_button = Button(
    self.connection_frame,
    text="Connect",
    command=self.connect_to_server
)
self.connect_button.pack(pady=20)

def create_chat_frame(self):
    """Create the chat interface"""
    self.chat_frame = Frame(self.root)

    # Chat display
    self.chat_display = scrolledtext.ScrolledText(
        self.chat_frame,
        state='disabled',
        wrap=WORD,
        width=60,
        height=20
    )
    self.chat_display.pack(pady=10, padx=10, fill=BOTH, expand=True)

    # Message entry
    self.message_entry = Entry(self.chat_frame, width=50)
    self.message_entry.pack(side=LEFT, padx=5, pady=5, fill=X,
expand=True)
    self.message_entry.bind("<Return>", self.send_message)

    # Send button
    self.send_button = Button(

```

```

        self.chat_frame,
        text="Send",
        command=self.send_message
    )
    self.send_button.pack(side=RIGHT, padx=5, pady=5)

    # Disconnect button
    self.disconnect_button = Button(
        self.chat_frame,
        text="Disconnect",
        command=self.disconnect,
        fg="red"
    )
    self.disconnect_button.pack(side=BOTTOM, pady=5)

def show_connection_frame(self):
    """Show the connection setup interface"""
    self.chat_frame.pack_forget()
    self.connection_frame.pack(fill=BOTH, expand=True)

def show_chat_frame(self):
    """Show the chat interface"""
    self.connection_frame.pack_forget()
    self.chat_frame.pack(fill=BOTH, expand=True)
    self.message_entry.focus_set()

def connect_to_server(self):
    """Connect to the messaging server"""
    server_ip = self.server_ip_entry.get()
    port = self.port_entry.get()
    self.username = self.username_entry.get().strip()

    if not self.username:
        messagebox.showerror("Error", "Please enter a username")
        return

    try:
        port = int(port)
        self.client_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        self.client_socket.connect((server_ip, port))

        # Send username to server
        connect_msg = {'username': self.username}
        self.client_socket.send(json.dumps(connect_msg).encode('utf-8'))

```

```

        self.connected = True
        self.show_chat_frame()

        # Start thread to receive messages
        receive_thread = threading.Thread(target=self.receive_messages)
        receive_thread.daemon = True
        receive_thread.start()

    except Exception as e:
        messagebox.showerror("Connection Error", f"Failed to connect:
{e}")

def disconnect(self):
    """Disconnect from the server"""
    if self.client_socket:
        try:
            self.client_socket.close()
        except:
            pass
    self.connected = False
    self.client_socket = None
    self.show_connection_frame()

def send_message(self, event=None):
    """Send a message to the server"""
    if not self.connected:
        return

    message = self.message_entry.get().strip()
    if not message:
        return

    try:
        message_data = {
            'type': 'message',
            'sender': self.username,
            'content': message
        }
        self.client_socket.send(json.dumps(message_data).encode('utf-8'))
        self.message_entry.delete(0, END)
    except Exception as e:
        print(f"Error sending message: {e}")
        self.disconnect()

def receive_messages(self):
    """Receive messages from server"""

```

```

while self.connected:
    try:
        message = self.client_socket.recv(1024).decode('utf-8')
        if not message:
            break

        message_data = json.loads(message)
        self.display_message(message_data)

    except Exception as e:
        print(f"Error receiving message: {e}")
        break

self.disconnect()

def display_message(self, message_data):
    """Display a message in the chat window"""
    self.chat_display.config(state='normal')

    if message_data['type'] == 'history':
        for msg in message_data['content']:
            self.format_and_insert(msg)
    else:
        self.format_and_insert(message_data)

    self.chat_display.config(state='disabled')
    self.chat_display.see(END)

def format_and_insert(self, message_data):
    """Format and insert a message into the chat display"""
    timestamp = datetime.strptime(
        message_data['timestamp'],
        '%Y-%m-%d %H:%M:%S.%f'
    ).strftime('%H:%M')

    if message_data['type'] == 'system':
        self.chat_display.insert(END, f"[{timestamp}] SYSTEM:
{message_data['content']}\n", 'system')
    else:
        self.chat_display.insert(END, f"[{timestamp}]
{message_data['sender']}: {message_data['content']}\n")

    # Configure tag for system messages
    self.chat_display.tag_config('system', foreground='blue')

if __name__ == "__main__":

```

```
root = Tk()
app = MessagingClient(root)
root.mainloop()
```

Features

1. Server Functionality:

- Handles multiple client connections simultaneously
- Maintains message history
- Broadcasts messages to all connected clients
- Tracks user join/leave events

2. Client Functionality:

- Graphical user interface with Tkinter
- Real-time message display
- Message history retrieval
- Connection management
- Username support

3. Message Types:

- Regular chat messages
- System notifications (user join/leave)
- Message history transfer

How to Use

1. Start the Server:

```
python messaging_server.py
```

2. Start Clients:

```
python messaging_client.py
```

- Enter server IP (localhost if running locally)
- Enter port (default 5555)
- Choose a username

3. Chat:

- Type messages and press Enter/Send button
- See messages from all connected users

Requirements

- Python 3.x

- No additional libraries required (uses built-in modules)

Possible Enhancements

1. Add user authentication
2. Implement private messaging
3. Add file sharing capability
4. Improve error handling and reconnection logic
5. Add message encryption
6. Implement a database for persistent message history

Would you like me to explain any specific part of the code in more detail or suggest any modifications?